

ROTEIRO DE LABORATÓRIO 3

Medindo linguagens: tipagem, execução e memória na prática

Aula 3, 25 de agosto · duração prevista: 2h30

O que você vai construir

Este roteiro troca opinião por medição. A mesma tarefa é escrita em Python e em JavaScript, o tempo é cronometrado, o comportamento diante de erro de tipo é registrado, e as conclusões são escritas a partir dos números obtidos.

Além da comparação, três experimentos reproduzem defeitos que aparecem em sistemas reais: dinheiro guardado em ponto flutuante, texto lido com a codificação errada, e memória que cresce sem parar mesmo com coletor de lixo ligado.

AO FINAL DESTES ROTEIRO VOCÊ TERÁ

Dois programas equivalentes, um em Python e um em JavaScript, com tempos medidos.

Um registro de como cada linguagem reage a um argumento do tipo errado.

Uma demonstração do erro de ponto flutuante e da correção com Decimal.

Um documento com a tabela de medições e a análise à luz dos três eixos.

Antes de começar

Nada além do que já foi instalado nos roteiros anteriores. A parte em JavaScript roda no console do navegador, sem instalar nada.

- Python 3.12 ou superior.
- Um navegador com console de desenvolvedor, como Chrome, Edge ou Firefox.
- O repositório da equipe clonado.

Passo a passo

Os passos 1 a 4 produzem os dados. Os passos 5 a 7 exploram os defeitos clássicos. O passo 8 consolida tudo em um documento.

1**Preparar a pasta** 5 min

NO TERMINAL

```
cd ads-2026-2-biblioteca
mkdir aula03-eixos
cd aula03-eixos
```

```
# conferir que o Python responde
python --version
```

SAÍDA ESPERADA

```
Python 3.12.4
```

POR QUE ISSO IMPORTA

A parte em JavaScript deste roteiro não usa Node.js. Ela roda no console do próprio navegador, que já está instalado na sua máquina.

Para abrir o console: no Chrome ou no Edge, tecla F12 e vá na aba Console. No Firefox, F12 e depois Console. É possível digitar e executar JavaScript ali mesmo.

CONFIRA ANTES DE SEGUIR

O Python responde à versão e você conseguiu abrir o console do navegador.

2

Escrever a versão em Python 20 min

A tarefa é somar os quadrados de todos os números de 1 até dez milhões. É um trabalho puramente numérico, escolhido justamente por não depender de disco nem de rede, o que tornaria a medição menos confiável.

```
BENCH.PY
import time

LIMITE = 10_000_000

def soma_quadrados(n):
    total = 0
    for i in range(1, n + 1):
        total += i * i
    return total

if __name__ == "__main__":
    tempos = []
    for rodada in range(3):
        inicio = time.perf_counter()
        resultado = soma_quadrados(LIMITE)
        fim = time.perf_counter()
        duracao = fim - inicio
        tempos.append(duracao)
        print(f"rodada {rodada + 1}: {duracao:.3f} s")

    tempos.sort()
    print(f"resultado: {resultado}")
    print(f"mediana: {tempos[1]:.3f} s")
```

NO TERMINAL

```
python bench.py
```

SAÍDA ESPERADA

```
rodada 1: 0.462 s
rodada 2: 0.447 s
rodada 3: 0.451 s
resultado: 33333338333335000000
mediana: 0.451 s
```

ATENÇÃO

Os tempos da sua máquina vão ser diferentes dos exemplos, e isso é esperado. O que importa é a comparação

entre as duas linguagens na MESMA maquina.

O resultado numerico, porem, deve ser exatamente 33333338333335000000. Se der outro numero, ha erro no laco.

POR QUE ISSO IMPORTA

Repare no tamanho do resultado: 333 quintilhoes. Em Java ou C, um inteiro comum de 64 bits nao guardaria esse valor sem estourar.

Python nao tem esse limite, porque seus inteiros crescem conforme a necessidade. E uma conveniencia que se paga em desempenho, e e parte da explicacao do tempo medido.

CONFIRA ANTES DE SEGUIR

O resultado numérico bate exatamente com o esperado e a mediana foi registrada.

3

Escrever a versão em JavaScript, no console do navegador 20 min

A mesma tarefa, agora em JavaScript. Não é preciso instalar nada: abra uma aba em branco no navegador, tecle F12, vá na aba Console e cole o código abaixo. Repare que a estrutura do programa é praticamente idêntica à versão em Python, o que é justamente o ponto.

COLE NO CONSOLE DO NAVEGADOR

```
const LIMITE = 10_000_000;

function somaQuadrados(n) {
  let total = 0n; // BigInt, para nao perder precisao
  for (let i = 1n; i <= BigInt(n); i++) {
    total += i * i;
  }
  return total;
}

const tempos = [];
let resultado;

for (let rodada = 0; rodada < 3; rodada++) {
  const inicio = performance.now();
  resultado = somaQuadrados(LIMITE);
  const fim = performance.now();
  const duracao = (fim - inicio) / 1000;
  tempos.push(duracao);
  console.log(`rodada ${rodada + 1}: ${duracao.toFixed(3)} s`);
}

tempos.sort((a, b) => a - b);
console.log(`resultado: ${resultado}`);
console.log(`mediana:   ${tempos[1].toFixed(3)} s`);
```

NO CONSOLE DO NAVEGADOR

cole o codigo e tecle Enter

SAÍDA ESPERADA

```
rodada 1: 0.631 s
rodada 2: 0.608 s
rodada 3: 0.615 s
```

```
resultado: 333333383333335000000
mediana: 0.615 s
```

ATENÇÃO

O resultado precisa ser o MESMO numero da versao em Python. Se for diferente, provavelmente o BigInt nao esta sendo usado.

Foi preciso usar BigInt porque o numero comum de JavaScript e um ponto flutuante de 64 bits, e ele perde precisao acima de 9 quadrilhoes. Esse detalhe e material de analise, nao um contratempo.

POR QUE ISSO IMPORTA

A diferenca entre as duas linguagens costuma ser bem menor do que a turma espera, e em muitas maquinas o JavaScript com BigInt fica ate mais lento que o Python. O motivo e que BigInt e um caminho lento do motor, fora da otimizacao automatica que o V8 aplica a numeros comuns.

Faca uma segunda medicao trocando BigInt por numero comum: apague os sufixos n e a conversao BigInt(n). Em uma das maquinas de referencia o tempo caiu de 0.615 s para 0.013 s, cerca de 47 vezes mais rapido, e o resultado saiu 333333383333717100000, que esta ERRADO.

Velocidade e precisao foram trocadas uma pela outra, e a linguagem obrigou voce a escolher. Registre os dois numeros no documento final.

CONFIRA ANTES DE SEGUIR

As duas linguagens produzem o mesmo número e você tem duas medianas para comparar. Salve o código em um arquivo bench.js na pasta do laboratório, para que ele fique registrado no repositório.

4

Provocar um erro de tipo em cada linguagem 20 min

Agora o experimento sobre tipagem. Chame a função passando um texto no lugar do número e registre exatamente o que cada linguagem faz.

TIPOS.PY

```
def total(preco, quantidade):
    return preco * quantidade

print("caso correto: ", total(19.90, 3))

print("caso errado: ", total(19.90, "3"))
```

NO TERMINAL

```
python tipos.py
```

SAÍDA ESPERADA

```
caso correto: 59.7
Traceback (most recent call last):
  File "tipos.py", line 7, in <module>
    print("caso errado: ", total(19.90, "3"))
  File "tipos.py", line 2, in total
    return preco * quantidade
TypeError: can't multiply sequence by non-int of type 'float'
```

TIPOS.JS, COLE NO CONSOLE DO NAVEGADOR

```
function total(preco, quantidade) {  
    return preco * quantidade;  
}  
  
console.log("caso correto: ", total(19.90, 3));  
console.log("caso errado:  ", total(19.90, "3"));  
console.log("caso pior:    ", total(19.90, "tres"));  
  
console.log("concatenacao: ", "3" + 1);  
console.log("subtracao:    ", "3" - 1);
```

NO CONSOLE DO NAVEGADOR

cole o código e tecle Enter

SAÍDA ESPERADA

```
caso correto:  59.699999999999996  
caso errado:   59.699999999999996  
caso pior:     NaN  
concatenacao:  31  
subtracao:     2
```

POR QUE ISSO IMPORTA

Compare as duas saídas com atenção. Python parou e apontou a linha exata do problema. JavaScript converteu o texto "3" para número e seguiu como se nada tivesse acontecido.

O caso pior é o mais instrutivo: com "tres" o resultado virou NaN, que significa não é um número, e o programa continuou rodando. Um valor NaN somado a outros valores contamina tudo em silêncio até aparecer na tela do usuário.

Registre no documento final qual dos dois comportamentos é preferível, e em que situação.

CONFIRA ANTES DE SEGUIR

Você registrou as duas saídas completas, incluindo a mensagem de erro do Python na íntegra.

5

O experimento do dinheiro 20 min

Este é o defeito mais caro da lista, e o mais fácil de reproduzir.

DINHEIRO.PY

```
from decimal import Decimal  
  
print("--- com float, o padrao ---")  
print(0.1 + 0.2)  
print(0.1 + 0.2 == 0.3)  
  
# simulando um caixa: mil vendas de dez centavos  
caixa = 0.0  
for _ in range(1000):  
    caixa += 0.10  
print(f"caixa apos 1000 vendas de R$0,10: {caixa}")  
print(f"deveria ser exatamente 100.0:      {caixa == 100.0}")  
  
print()  
print("--- com Decimal ---")  
print(Decimal("0.1") + Decimal("0.2"))
```

```
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

caixa = Decimal("0.00")
for _ in range(1000):
    caixa += Decimal("0.10")
print(f"caixa com Decimal:          {caixa}")
print(f"deveria ser exatamente 100.00: {caixa == Decimal('100.00')}")
```

NO TERMINAL
python dinheiro.py

SAÍDA ESPERADA

```
--- com float, o padrao ---
0.300000000000000004
False
caixa apos 1000 vendas de R$0,10: 99.9999999999986
deveria ser exatamente 100.0:      False

--- com Decimal ---
0.3
True
caixa com Decimal:          100.00
deveria ser exatamente 100.00: True
```

ATENÇÃO

Repare no valor do caixa: mil vendas de dez centavos deram 99,9999999999986 em vez de 100. O erro é minúsculo por operação e se acumula. Em um sistema com milhões de transações, essa diferença aparece na conciliação contábil e vira problema de auditoria. E por isso que sistema financeiro não guarda dinheiro em ponto flutuante.

POR QUE ISSO IMPORTA

Note que Decimal recebe o valor como TEXTO: Decimal("0.1"), e não Decimal(0.1). Se você passar um float, o erro já veio embutido antes da conversão. A alternativa mais simples, usada em muitos sistemas, é guardar tudo em centavos como número inteiro, e dividir por 100 apenas na hora de exibir.

CONFIRA ANTES DE SEGUIR

A saída mostra a diferença entre float e Decimal nos quatro casos.

6

O experimento da acentuação 20 min

Texto também é um tipo, e a regra que converte caracteres em bytes se chama codificação. Este experimento reproduz um defeito que quase todo sistema brasileiro já teve.

TEXTO.PY

```
nomes = "Joao da Silva\nConceicao Araujo\nAntonio Coracao\n"
nomes = nomes.replace("Joao", "João")
nomes = nomes.replace("Conceicao", "Conceição")
nomes = nomes.replace("Araujo", "Araújo")
nomes = nomes.replace("Antonio", "Antônio")
nomes = nomes.replace("Coracao", "Coração")
```

```
# gravar em UTF-8, que e o padrao moderno
with open("clientes.txt", "w", encoding="utf-8") as f:
    f.write(nomes)

print("--- lido corretamente, como UTF-8 ---")
with open("clientes.txt", "r", encoding="utf-8") as f:
    print(f.read())

print("--- lido com a codificacao errada ---")
with open("clientes.txt", "r", encoding="latin-1") as f:
    print(f.read())
```

NO TERMINAL
python texto.py

SAÍDA ESPERADA

```
--- lido corretamente, como UTF-8 ---
João da Silva
Conceição Araújo
Antônio Coração

--- lido com a codificacao errada ---
JoÃ£o da Silva
ConceiÃ§Ã£o AraÃºjo
AntÃ´nio CoraÃ§Ã£o
```

POR QUE ISSO IMPORTA

Nenhum erro foi levantado na segunda leitura. O programa considerou que fez o seu trabalho, e o dado saiu errado em silêncio.

E a mesma categoria de problema do NaN em JavaScript: a linguagem preferiu seguir a falhar, e o custo disso apareceu depois.

A prevencao e sempre declarar a codificacao ao abrir arquivo, e usar UTF-8 em tudo: arquivo, banco de dados e resposta de API.

CONFIRA ANTES DE SEGUIR

As duas leituras aparecem na tela, uma correta e uma corrompida.

7

O experimento da memória 20 min

Python tem coletor de lixo automático, e mesmo assim vaza memória quando o código mantém referência para dados que não usa mais. Este experimento mostra o efeito.

```
MEMORIA.PY
import sys

CACHE = {}

def consultar_cliente(cpf):
    """Simula uma consulta cara, guardando o resultado."""
    if cpf not in CACHE:
        CACHE[cpf] = {"cpf": cpf, "dados": "x" * 1000}
    return CACHE[cpf]
```

```

for i in range(50_000):
    consultar_cliente(f"cpf-{i}")

total_bytes = sys.getsizeof(CACHE)
for chave, valor in CACHE.items():
    total_bytes += sys.getsizeof(chave) + sys.getsizeof(valor["dados"])

print(f"itens no cache: {len(CACHE)}")
print(f"memoria aproximada: {total_bytes / 1_048_576:.1f} MB")
print("nada nunca sai do cache")

```

NO TERMINAL
python memoria.py

SAÍDA ESPERADA
itens no cache: 50000
memoria aproximada: 55.3 MB
nada nunca sai do cache

A correção

```

MEMORIA_CORRIGIDA.PY
from functools import lru_cache

@lru_cache(maxsize=1000)
def consultar_cliente(cpf):
    return {"cpf": cpf, "dados": "x" * 1000}

for i in range(50_000):
    consultar_cliente(f"cpf-{i}")

info = consultar_cliente.cache_info()
print(info)
print(f"itens guardados agora: {info.currsize}")

```

NO TERMINAL
python memoria_corrigida.py

SAÍDA ESPERADA
CacheInfo(hits=0, misses=50000, maxsize=1000, currsize=1000)
itens guardados agora: 1000

POR QUE ISSO IMPORTA

O coletor de lixo nao resolveu o primeiro caso porque, do ponto de vista dele, todos os objetos continuavam alcançáveis a partir da variável global CACHE.

O vazamento era logico, e nao tecnico. Nenhum mecanismo automatico consegue adivinhar que aquele dado deixou de ser necessario: essa decisao e do programador, e no segundo caso ela foi declarada com o maxsize.

CONFIRA ANTES DE SEGUIR

A versão com lru_cache mantém 1000 itens em vez de 50 mil.

Crie o arquivo COMPARATIVO.md reunindo tudo. A parte que vale é a interpretação, não a coleta.

ESTRUTURA DO COMPARATIVO.MD

Comparativo Python x JavaScript

Máquina usada

Processador, memória RAM e sistema operacional.

Tempos medidos (mediana de 3 rodadas)

Linguagem	Mediana	Resultado
Python	__ s	_____
JavaScript	__ s	_____

Comportamento diante de erro de tipo

O que cada linguagem fez ao receber texto no lugar de número.

Os três experimentos

- Dinheiro: o que aconteceu com float e o que mudou com Decimal
- Acentuação: por que nenhum erro foi levantado
- Memória: por que o coletor de lixo não resolveu

Análise (três parágrafos)

1. Qual dos três eixos explica melhor a diferença de tempo medida?
2. Qual comportamento diante de erro de tipo a equipe prefere, e por que?
3. Que decisão concreta a equipe vai tomar no projeto do semestre por causa do que foi observado aqui?

NO TERMINAL

```
git add .
git commit -m "adiciona medicoes comparativas e experimentos dos tres eixos"
git push
```

ATENÇÃO

O terceiro parágrafo da análise é o que será lido com mais atenção. Uma decisão concreta é algo como: valores monetários do projeto serão guardados em centavos como inteiro, ou todo arquivo será aberto declarando encoding utf-8.

Resposta genérica do tipo vamos tomar mais cuidado não conta como decisão.

CONFIRA ANTES DE SEGUIR

O COMPARATIVO.md está no repositório com a tabela preenchida e os três parágrafos escritos.

Se der errado

Os problemas abaixo respondem pela maior parte das travadas neste roteiro.

Sintoma	O que fazer
Nao acho o console do navegador	No Chrome e no Edge, tecle F12 e escolha a aba Console. Se F12 não funcionar, use o menu, Mais ferramentas, Ferramentas do desenvolvedor.
O navegador avisa que a pagina travou	É esperado: o laço segura a aba por alguns segundos. Aguarde, ou reduza o LIMITE para 1 milhão nas duas linguagens.
SyntaxError com 10_000_000	Navegador ou Python antigos. Use 10000000 sem o separador.
Cannot mix BigInt and other types	Faltou o sufixo n em algum literal, ou a conversão BigInt(n). Confira a linha do laço.
O console recusa colar o código	Alguns navegadores pedem que você digite a palavra allow pasting antes de permitir colar no console. Faça isso uma vez e o bloqueio some.
UnicodeDecodeError no texto.py	Esse erro só aparece em algumas combinações. Se acontecer, troque latin-1 por cp1252, que produz o mesmo efeito didático.
Os tempos variam muito entre rodadas	Feche navegador e outros programas pesados. Use a mediana das três rodadas, que já é o que o script calcula.

Desafios opcionais

Para quem terminou antes do fim do laboratório. Não valem nota, mas costumam aparecer na prova em alguma forma.

- Reescreva o bench.py usando sum com uma expressão geradora, em vez do laço explícito, e meça de novo. A diferença surpreende.
- Descubra por que o número comum de JavaScript perde precisão acima de 9.007.199.254.740.991, e o que esse valor tem de especial. A resposta está na documentação de Number.MAX_SAFE_INTEGER.
- Descubra o que é o interpretador PyPy, rode o bench.py nele se conseguir instalar, e compare com o CPython padrão.
- Escreva a mesma soma de quadrados em uma terceira linguagem, à escolha da equipe, e acrescente a linha na tabela.
- Investigue por que 0.1 e 0.2 não têm representação binária exata, mas 0.5 e 0.25 têm.

Entrega

O que entregar	A pasta aula03-eixos com bench.py, bench.js, tipos.py, tipos.js, dinheiro.py, texto.py, memoria.py e COMPARATIVO.md. Os dois arquivos .js ficam salvos apenas como registro, já que foram executados no navegador
Onde entregar	No repositório da equipe, com commit feito ainda em aula
Quem entrega	A equipe, com commits de todos os integrantes
Prazo	Até o final do encontro

Critério de aceite

- Os dois programas produzem o mesmo resultado numérico.
- A tabela do COMPARATIVO.md tem os tempos medidos na máquina da equipe, e não os do exemplo.

- As saídas dos experimentos foram registradas como apareceram, inclusive as mensagens de erro.
- A análise responde às três perguntas, e a terceira traz uma decisão concreta para o projeto.

Para consultar durante o laboratório

The Floating-Point Guide, o que todo programador deveria saber

<https://floating-point-gui.de/>

Python, aritmética de ponto flutuante, problemas e limitações

<https://docs.python.org/pt-br/3/tutorial/floatingpoint.html>

Python, documentação do módulo decimal

<https://docs.python.org/pt-br/3/library/decimal.html>

Python, documentação de functools.lru_cache

<https://docs.python.org/pt-br/3/library/functools.html>

MDN Web Docs, BigInt em JavaScript

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/BigInt

Chromium, Memory safety

<https://www.chromium.org/Home/chromium-security/memory-safety/>